CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA DE COMPUTAÇÃO – SISTEMAS DISTRIBUÍDOS

Transferência de arquivos Peer-to-Peer

Trabalho Prático 2 – Professora Michelle Hanne

Pedro Augusto de Portilho Ronzani
Lucas Andrade Brandão

Belo Horizonte

Código-fonte: https://github.com/LucasAbrandao/sd-tp2-p2p

1 Objetivo e arquitetura

O trabalho implementa um sistema elementar de transferência de arquivos por blocos, no qual cada nó (*peer*) atua simultaneamente como cliente e servidor, conforme o enunciado [1]. O objetivo é recuperar o arquivo a partir de vizinhos estáticos, compartilhar blocos já recebidos e verificar a integridade do conteúdo remontado. Um nó com o arquivo completo é o *seeder*; os demais são *leechers* durante o download.

A solução utiliza Python 3.10 ou superior e apenas sua biblioteca padrão. A comunicação é realizada por sockets TCP não bloqueantes com `asyncio`. Cada processo recebe uma configuração JSON com identificador, IP, porta, vizinhos, manifesto, diretório de saída e, quando aplicável, arquivo de origem. O manifesto é distribuído previamente; não há *tracker*.

A organização do código separa as responsabilidades apresentadas na Tabela 1.

Tabela 1 – Organização da implementação

Módulo	Responsabilidade
`metadata.py`	Manifesto, fragmentação lógica e hashes.
`protocol.py`	Enquadramento e serialização das mensagens TCP.
`storage.py`	Validação, persistência, retomada e montagem.
`peer.py`	Servidor, clientes concorrentes e registros de eventos.
`__main__.py`	Interface de execução e geração do manifesto.

Fonte: código-fonte do projeto [2].

O servidor inicia antes das tarefas de download, uma por vizinho. As tarefas consultam inventários e reservam índices para evitar solicitações duplicadas locais. A seleção percorre índices em ordem circular, com início derivado do identificador do nó, e atualiza o inventário remoto após lotes de até 32 pedidos. Durante a transferência, as operações de disco e a montagem são executadas em threads para liberar o laço de eventos. Um bloco validado e salvo já pode ser servido; concluir o download não encerra o servidor.

2 Metadados, protocolo e armazenamento

O manifesto JSON contém cinco campos: `name`, `size`, `block_size`, `sha256` e `blocks`. Este último armazena, em ordem, o SHA-256 de cada bloco; o campo `sha256` contém o hash do arquivo completo. Para um arquivo de L bytes e blocos de B bytes, a quantidade de blocos é $\lceil L/B \rceil$, e o último pode ser menor que B. O seeder lê cada trecho diretamente do arquivo original, sem exigir uma cópia física prévia por bloco.

O identificador `file_id` é o SHA-256 de uma representação canônica do manifesto. Antes de trocar blocos, os nós conferem versão do protocolo e esse identificador, evitando combinar transferências com metadados incompatíveis. A Tabela 2 apresenta as mensagens utilizadas.

Tabela 2 – Mensagens do protocolo

Mensagem	Finalidade
`hello`	Conferir a versão e o `file_id` e identificar o peer.
`status`	Consultar ou informar os índices disponíveis.
`get`	Solicitar o bloco identificado por um índice.
`block`	Entregar índice e conteúdo codificado em Base64.
`missing`	Informar a indisponibilidade do bloco solicitado.

Fonte: código-fonte do projeto [2].

Cada mensagem possui um prefixo de quatro bytes, sem sinal e em ordem de rede, com o tamanho do corpo JSON em UTF-8. A leitura usa `readexactly` para tratar fragmentação ou concatenação no fluxo TCP. Quadros vazios, maiores que 2 MiB ou com conteúdo inválido são rejeitados. Há prazos para conexão, leitura, escrita e conclusão do download.

2.1 Integridade, persistência e recuperação

O inventário local utiliza conjuntos de índices. Antes de aceitar um bloco, o armazenamento verifica índice, tamanho e SHA-256. A gravação ocorre em arquivo temporário, seguido de substituição atômica; somente depois o índice fica disponível para compartilhamento. Os blocos persistem em `.blocks/<file_id>/<indice>.part` e são revalidados ao reiniciar o nó, permitindo retomar somente o conteúdo íntegro.

A montagem segue a ordem dos índices e confere novamente os blocos, o tamanho total e o SHA-256 final. O arquivo é sincronizado com `fsync` e publicado por um *hard link*, sem sobrescrever um arquivo de conteúdo diferente já existente no destino. Portanto, o sistema de arquivos de saída deve suportar esse recurso. Não há `fsync` individual por bloco.

Falhas de comunicação liberam reservas e permitem reconexão; mensagens inválidas encerram a conexão afetada. Leituras para envio também revalidam o conteúdo. Os logs JSON registram índice, origem ou destino e progresso, permitindo acompanhar o recebimento e o compartilhamento dos blocos.

3 Procedimento experimental e validação

A avaliação principal foi executada em 22/10/2026, em um Apple M5 com 16 GiB de RAM, macOS 27.0 (build 26A428) e Python 3.12.14. Cada peer foi um processo independente, comunicando-se por TCP em `127.0.0.1`. O procedimento e os resultados detalhados estão preservados no repositório [3].

A matriz combina 2 e 4 peers, blocos de 1.024 e 4.096 bytes e arquivos de 10/20 KiB, 1/5 MiB e 10/20 MiB, totalizando 24 cenários. Adotam-se unidades binárias: 1 KiB = 1.024 bytes e 1 MiB = 1.048.576 bytes. Cada cenário usa um arquivo aleatório novo, criado com `os.urandom`, e diretórios próprios. Há uma execução por combinação.

Com dois peers, A é o seeder e B o receptor. Com quatro, A conhece B/C, B e C conhecem A/D, e D conhece apenas B/C. A Figura 1 ilustra essa configuração, que obriga D a obter os blocos dos intermediários.

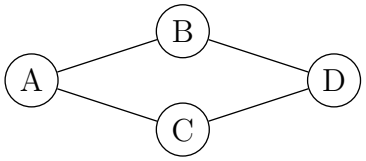


Figura 1 – Vizinhança estática dos cenários com quatro peers
Fonte: configuração do benchmark do projeto [2].

Os receptores iniciam antes da fonte. Nos cenários de quatro peers com arquivos de 10 e 20 KiB, aplica-se atraso artificial de 3 ms por bloco para tornar o envio parcial observável. Nos demais, o atraso é zero. O tempo total é medido desde o início do primeiro processo até a conclusão de todos os leechers, incluindo inicialização e montagem; exclui a preparação do arquivo e do manifesto.

3.1 Testes automatizados e critérios de aprovação

Os 28 testes passaram, incluindo quatro integrações com processos reais. A suíte cobre manifesto, blocos inválidos, retomada, arquivo vazio, último bloco parcial, destino existente, enquadramento TCP, timeouts, mensagens malformadas, compartilhamento parcial e chegada tardia da fonte. No macOS, corrigiu-se apenas a expectativa de caminhos no teste: `directory` e `seed` passaram a usar `resolve()`, assim como a configuração, considerando a equivalência entre `/var` e `/private/var`. O código de transferência não mudou.

Para aprovar um cenário, o benchmark exige conclusão, tamanho e SHA-256 iguais aos da origem. Com quatro peers, exige ainda que D receba somente de B/C e que haja envio para D com `have < total` no emissor. A auditoria posterior recalculou hashes em disco e conferiu os índices recebidos nos logs completos.

4 Resultados no macOS

Todos os 24 cenários foram aprovados. Os 48 arquivos baixados pelos leechers possuíam tamanho e SHA-256 corretos. O benchmark registra 72 verificações: 24 fontes e 48 downloads. A auditoria posterior verificou 96 arquivos, incluindo também as 24 saídas montadas pelos seeders. A Tabela 3 reúne os tempos medidos.

Tabela 3 – Tempo de conclusão por cenário no macOS, em segundos

	2 peers		4 peers	
Arquivo	**1 KiB**	**4 KiB**	**1 KiB**	**4 KiB**
10 KiB	0,075	0,088	0,195	0,189
20 KiB	0,087	0,087	0,215	0,171
1 MiB	0,676	0,228	0,835	0,335
5 MiB	3,114	0,847	3,104	0,979
10 MiB	6,409	1,640	7,742	1,983
20 MiB	13,681	3,228	16,012	3,889

Fonte: execução macOS, `results.csv` [3].

Nos arquivos de 1 a 20 MiB, blocos de 4 KiB reduziram o tempo em todas as configurações. Em 20 MiB, a razão entre os tempos com blocos de 1 e 4 KiB foi 4,24 com dois peers e 4,12 com quatro. Nesse tamanho, o número de blocos cai de 20.480 para 5.120. A tendência é compatível com menor custo de solicitações, hashes, logs e operações de arquivo; o experimento não isola cada custo.

4.1 Compartilhamento antes da conclusão

Nos 12 cenários com quatro peers, D recebeu exclusivamente de B/C e houve envio antes da conclusão do emissor. Foram observados 45.346 desses eventos, distribuídos conforme a Tabela 4.

Tabela 4 – Envios de B/C para D com inventário ainda incompleto

Arquivo	**Blocos de 1 KiB**	**Blocos de 4 KiB**
10 KiB	6	1
20 KiB	17	3
1 MiB	1.014	247
5 MiB	5.035	1.237
10 MiB	10.216	2.552
20 MiB	19.989	5.029

Fonte: eventos `block_sent` da execução macOS [3].

A matriz totalizou 184.472 eventos de recebimento e 184.472 de envio. Cada leecher registrou todos os índices esperados uma única vez. Houve 684 tentativas de conexão recusadas, com novas tentativas, e duas conexões reiniciadas pelo outro peer. Todos os cenários terminaram com arquivos íntegros, e nenhum dos 72 processos produziu mensagens em `stderr`. Os eventos permanecem nos logs.

5 Comparação, limitações e conclusão

A execução original do repositório, em Windows 11 e Python 3.13, datada de 18/10/2026, também registrou 24/24 cenários aprovados e compartilhamento parcial nos 12 cenários com quatro peers. A Tabela 5 apresenta um recorte dessa comparação. Em ambos os ambientes, blocos de 4 KiB foram mais rápidos nos arquivos de 1 a 20 MiB.

Tabela 5 – Comparação descritiva entre execuções, em segundos

Cenário (arquivo; peers; bloco)	Windows	macOS
1 MiB; 2 peers; 1 KiB	5,855	0,676
20 MiB; 2 peers; 1 KiB	119,491	13,681
20 MiB; 4 peers; 1 KiB	206,628	16,012
20 MiB; 4 peers; 4 KiB	38,916	3,889

Fonte: matriz original e execução macOS [3].

A razão entre os tempos Windows e macOS variou de 8,18 a 12,90 nos 24 pares. A comparação é descritiva: hardware, sistema, versão do Python e condições de execução diferem, sem isolamento de suas contribuições.

5.1 Limitações e síntese

Há uma execução por cenário, sem médias ou medidas de dispersão. Os peers compartilham CPU e disco, com TCP em loopback. Quatro peers envolvem três downloads e topologia distinta de dois peers, que envolvem um. Os menores arquivos com quatro peers usam atraso artificial. Esses fatores impedem conclusões causais sobre escalabilidade ou desempenho em rede física.

A solução pressupõe manifesto confiável e sistema de arquivos com hard links. Não oferece autenticação, criptografia ou descoberta dinâmica. A retomada revalida os dados, sem sincronização física individual por bloco.

Nos cenários avaliados, verificaram-se transferência íntegra, cliente e servidor simultâneos, remontagem e redistribuição durante o download. A reprodução em dois ambientes reforça a evidência funcional; não identifica a causa da diferença de tempos.

Reprodução. Executar `python -m unittest discover -s tests -v` e `python scripts/benchmark.py -timeout 600 -output evidence/nova-execucao`. Os dados macOS estão em `evidence/macos-2026-10-22/`; os originais, em `evidence/`. Os logs integrais estão em `.runtime/benchmark-macos-2026-10-22/`, fora do versionamento.

Referências

[1] HANNE, Michelle. *Trabalho Prático 2: Transferência de Arquivos Peer-to-Peer*. Sistemas Distribuídos, CEFET-MG, 2026/2. Enunciado.

[2] BRANDÃO, L. A.; RONZANI, P. A. P. *sd-tp2-p2p: implementação e scripts de teste*. 2026. Disponível em: https://github.com/LucasAbrandao/sd-tp2-p2p.

[3] BRANDÃO, L. A.; RONZANI, P. A. P. *Evidências experimentais do TP2*. Matrizes Windows (18 out. 2026) e macOS (22 out. 2026), diretório `evidence/` do repositório.